



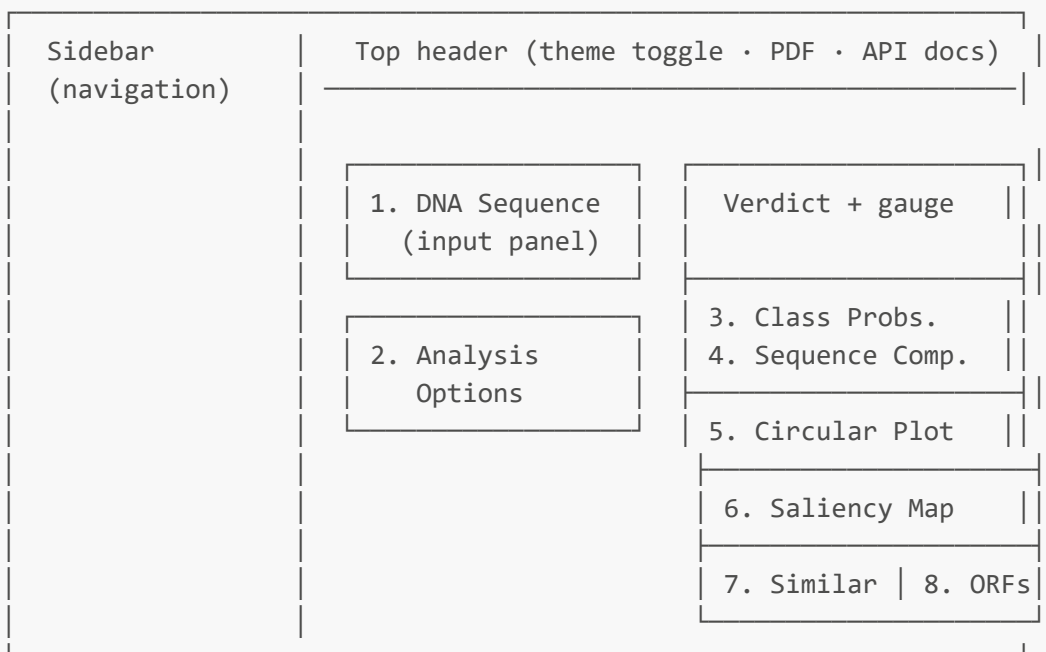
Analyze Page — Complete Component Guide

A deep-dive into every panel, card, and control on the **Analyze** page of GenomelQ. For each component this document covers:

- 🎯 **What it is** (plain-language description)
- 🗨️ **What it tells you** (how to read the output)
- ⚙️ **How it works under the hood** (the algorithm)
- 📖 **How to use it** (the user flow)
- 💡 **Practical tips** (when it's useful, when to be cautious)



Page Layout Overview



The page is a 12-column responsive grid: **left 5 columns = input**, **right 7 columns = results**. On mobile both stack vertically.

1. 🧬 DNA Sequence (Input Panel)

🎯 What it is

The text-input panel where you paste, type, or load a DNA sequence to analyze.

🗨️ What it tells you

While you type, three live indicators update in real time:

Indicator	Meaning
Length	Number of valid DNA bases entered (after stripping whitespace and invalid characters).
GC%	Percentage of G + C bases. GC content is biologically meaningful — coding regions and CpG islands tend to be GC-rich.
Valid	Green "Yes" if length ≥ 20 bases; red "No" otherwise.

⚙️ How it works under the hood

JavaScript runs `computeLiveStats()` in `frontend/static/app.js`:

```
const cleaned = sequence.toUpperCase().replace(/^[^ACGTN]/g, "");
let g = 0, c = 0;
for (const ch of cleaned) {
  if (ch === "G") g++;
  else if (ch === "C") c++;
}
gc = ((g + c) / cleaned.length) * 100;
```

A debounce of 80 ms prevents redundant updates while you type. The check `length >= 20` matches the backend's `MIN_SEQUENCE_LENGTH` constant in `src/core/config.py`.

📖 How to use it

1. **Paste** any DNA sequence (whitespace and case are tolerated).
2. **Or click a sample button:**
 -  *BRCA1 (Cancer)* — a known tumour-suppressor fragment
 -  *INS (Diabetes)* — the human insulin gene
 -  *APP (Alzheimer's)* — amyloid precursor protein
 -  *ACTB (Normal)* — beta-actin housekeeping gene
3. **Click Clear** to start over.
4. **Click Analyze Sequence** to send the sequence to the backend.

💡 Practical tips

- **Whitespace and case don't matter** — `atg cat\nGATG` becomes `ATGCATGATG`.
- The accepted alphabet is `A T G C N` (where `N` = "any/unknown"). Anything else is silently stripped on the backend.
- **Minimum length: 20 bases.** Short queries ($< \sim 150$ bases) tend to give lower confidence because there are fewer k-mer features to match.
- **Maximum supported: 200,000 bases** (defined in `src/core/config.py`). Above that the request is rejected.
- The textarea is a `monospace` font with vertical resize handle so you can comfortably read longer sequences.

2. ⚙️ Analysis Options (Configuration Panel)

What it is

A small card with a model-backbone selector and three optional analysis toggles.

What it tells you

Model backbone (segmented pill)

Three radio-style options:

Backbone	What it does	When to pick
TFIDF (default)	Classical TF-IDF on 4-grams of 6-mers, with a soft-voting ensemble of Random Forest + Calibrated Linear SVM + Logistic Regression. Highest accuracy on this dataset (CV macro F1 = 0.96).	Default for everyday use. Fastest (~50 ms).
DNABERT-2	Fine-tuned 117 M-parameter genomic transformer. Captures long-range patterns better than k-mers. Slower (~1 sec on GPU).	When you want a learned, contextual representation. Good for novel sequences.
Ensemble	Averages the probabilities of TFIDF and DNABERT-2. The most robust verdict.	Use this when you want maximum confidence for a defense or report.

If a backbone is **disabled (gray)**, you haven't trained it yet — see [HOW_TO_RUN_IN_VSCODE.md](#) to enable it.

Saliency explanation (checkbox)

When **on**, the backend returns per-base contribution scores so the UI can render the colored Saliency Map (#6 below). When **off**, the explanation card is omitted (faster response).

Out-of-distribution check (checkbox)

When **on**, runs a cosine-similarity comparison against the training corpus to flag inputs the model has never seen anything like. Adds ~30 ms.

Similar sequence search (checkbox)

When **on**, returns the top-K nearest known sequences. Adds ~30 ms.

How it works under the hood

The toggles are passed to the API as JSON fields on **POST /predict**:

```
{
  "sequence": "...",
  "model": "tfidf",
  "include_explain": true,
  "include_ood": true,
```

```
"include_similar": true,
"top_k_similar": 5
}
```

In `src/api/service.py` → `predict_payload()`, each flag conditionally calls the corresponding analysis module (`src/ml/explain.py`, `ood.py`, `similarity.py`).

How to use it

- **First time / quick test:** leave defaults (TFIDF + all three on).
- **Comparing models:** run the same sequence with TFIDF, then DNABERT-2, then Ensemble. The verdict card shows the active backbone in a small badge.
- **Speed over depth:** turn off saliency and similarity for the fastest possible prediction.

Practical tips

- Toggling options does **not** automatically re-run the analysis — you have to press **Analyze Sequence** again.
- The **model badge** in the verdict card always shows which backbone produced the result, so you can compare runs.
- TFIDF is the most accurate backbone right now because the training set is small (~22k augmented samples, ~500 unique parents). DNABERT-2 will start beating it once you fine-tune for more epochs.

3. Class Probabilities (Bar Card)

What it is

A vertical bar chart showing how the model split its confidence across the four disease classes.

What it tells you

Each row is one disease class with:

- A class name and color badge
- A horizontal progress bar with width proportional to probability
- The exact percentage on the right

The class with the highest bar matches the verdict. The shape of the distribution tells you how confident the model is:

Shape	Meaning
One bar near 100%, others near 0%	Model is very sure
Two bars roughly equal	Model is torn between two diseases — interpret cautiously
All four bars roughly 25%	Model has no idea — check the OOD banner

How it works under the hood

The probabilities come from the classifier's `predict_proba()` method:

- For TFIDF: soft voting averages the calibrated probabilities of RF, SVM, and LR.
- For DNABERT-2 fine-tuned: a softmax over the model's logits.
- For Ensemble: average of the two above.

The values are in the range [0, 1] and always sum to 1.

How to use it

After running an analysis, **read top-down** — bars are sorted by probability descending. Hover over a bar isn't supported but the percentage is always visible.

Practical tips

- A confidence above **70%** is generally trustworthy if OOD risk is low.
- A confidence below **45%** with **HIGH OOD risk** means the model is uncertain — treat the verdict as a hint, not a conclusion.
- Compare the highest bar against the second-highest. A gap less than 10% means the two classes share many k-mer features — biologically plausible (e.g., Diabetes and Alzheimer's both involve metabolic pathways).

4. Sequence Composition (Doughnut Chart)

What it is

A Chart.js doughnut showing the distribution of A/T/G/C/N bases in your sequence, plus two summary statistics below it.

What it tells you

The doughnut

Five colored segments represent the count of each base in your sequence:

-  **A** (adenine)
-  **T** (thymine)
-  **G** (guanine)
-  **C** (cytosine)
-  **N** (unknown / ambiguous)

The two stat cards below

Stat	Meaning	Range
Entropy	Shannon entropy in bits per base. Random sequences max out near 2.0; repetitive ones approach 0.	0 – 2
Complexity	Linguistic complexity = (distinct 4-mers / total 4-mers). 1.0 = every 4-mer is unique; 0.5 = lots of repeats.	0 – 1

⚙️ How it works under the hood

Backend (`src/core/sequence.py` → `compute_stats()`):

```
counts = Counter(sequence)
entropy = -sum((p * log2(p)) for p in (count/length))
kmers = [seq[i:i+4] for i in range(len-3)]
complexity = len(set(kmers)) / max(len(kmers), 1)
is_low_complexity = complexity < 0.5 or entropy < 1.5
```

Frontend (`renderCompositionChart()` in `app.js`) feeds the counts into Chart.js with theme-aware colors (white border in light mode, dark in dark mode).

📖 How to use it

After analysis, glance at the doughnut to **eyeball biases**:

- A roughly even split (each base ~25%) suggests random / non-coding DNA.
- Heavy GC dominance (G + C > 60%) hints at a CpG island or coding region.
- Heavy AT dominance suggests intronic / regulatory regions.

💡 Practical tips

- **Low complexity (red flag)**: if `complexity_score < 0.5`, the sequence has many repeats — the classifier may produce unreliable verdicts. Often pairs with HIGH OOD risk.
- **Entropy near 0**: pure repeats like `AAAAA...` — prediction will be a hallucination, OOD will scream.
- **N content > 5%**: lots of "unknown" bases. Most k-mers containing N won't match training vocabulary, so confidence drops.

5. 🌀 Circular Sequence Map (Plotly Polar Plot)

🎯 What it is

A circular visualization of your DNA sequence, packing three independent layers of information into one compact view.

💬 What it tells you

Three concentric rings, plus a center text:

■	Outer ring – GC content profile	→ blue line tracing GC % in a sliding window
■	Middle ring – saliency hotspots	→ colored markers (slate→amber→rose)
■	Inner ring – forward (+) ORFs	→ green arcs
■	Inner ring – reverse (-) ORFs	→ purple arcs



The angle around the circle represents **position along the sequence** (0° = position 0, going clockwise). Tick labels at 0° , 90° , 180° , 270° show the corresponding base position.

⚙️ How it works under the hood

JavaScript `renderCircularPlot()` builds three Plotly traces:

1. **GC profile**: sliding window of size `max(20, len/200)`, stride = win/2. Each window's (G+C)/win is mapped to a radius between 0.85 and 1.0.
2. **Saliency hotspots**: subsamples the per-base saliency to ~720 markers; each marker's color is mapped through the slate→amber→rose gradient and its size grows with the score.
3. **ORFs**: each ORF in the response becomes an arc. Forward strand at radius 0.55 (green), reverse strand at radius 0.45 (purple). Arc length = ORF span as a fraction of total sequence.

The center annotation reads the predicted class and confidence directly from the prediction payload.

📖 How to use it

1. **Look at the outer ring** to see GC-rich vs GC-poor regions. A peak suggests a possible coding region or CpG island.
2. **Check the middle ring** — clusters of bright (rose) markers identify "hot zones" the model paid attention to. These often align with known gene motifs.
3. **Inspect ORFs** — long arcs mean credible protein-coding stretches. Hover over any arc to see strand, frame, and protein length.
4. **Cross-reference**: a saliency hot zone overlapping a long ORF is a high-quality biological signal — the model's reasoning aligns with the structural feature.

💡 Practical tips

- Plotly's built-in zoom is **not enabled** here on purpose (the chart is small). For deep inspection, look at the saliency map (#6) and ORF table (#8) which give exact numbers.
- The plot **respects dark mode** — angular gridlines and tick labels recolor automatically.
- This visualization is one of the project's flagship features for **presentations** — open the Presentation page slide 7 to see how it's introduced to an audience.

6. 🎨 Saliency Map (DNA Heatmap)

🎯 What it is

The actual nucleotides of your sequence printed out in a monospace font, with each base background-colored by how strongly it influenced the prediction.

💬 What it tells you

The colored bases

```

slate  _____  amber  _____  rose
low                                     high
(0.0)                                (1.0)

```

Each base shows a normalized contribution score in the range [0, 1]. Higher scores = the model relied more on the k-mers covering that position.

Top Regions chips

Below the map, up to 5 chips highlight the **highest-scoring contiguous regions**, e.g.:

```

50-80 • score 0.91
112-142 • score 0.85
...

```

These are the candidate "important motifs" — the regions you'd zoom into if you were biologically validating the prediction.

⚙️ How it works under the hood

`src/ml/explain.py` → `saliency()`:

1. Tokenize the sequence into overlapping 6-mers.
2. For each n-gram window of k-mers (here n=4) that appears in the trained vocabulary, look up the Random Forest's `feature_importances_` for the predicted class.
3. Distribute the importance back to every base position covered by that n-gram.
4. Normalize the per-base scores to [0, 1] using max-scaling.
5. Find the top regions via a sliding-window-mean argmax with non-maximum suppression.

Frontend (`renderSaliency()` in `app.js`) interpolates each score through a 3-stop gradient:

```

if (s < 0.5) blend(slate, amber, s/0.5)
else       blend(amber, rose, (s-0.5)/0.5)

```

The text color flips dark/light depending on the background brightness for legibility.

📖 How to use it

1. Run a prediction.
2. Scroll to the Saliency Map card.
3. Read the colored bases — bright clusters reveal the model's attention.
4. Check Top Regions chips for the precise position ranges.
5. Compare against known gene motifs in the literature to validate.

💡 Practical tips

- **Long sequences wrap into blocks of 60 chars** for readability, like a FASTA-style display.
- **Mostly slate / cool colors** → **low overall contribution**. Often happens with OOD inputs.
- **A few intense bright clusters** → **strong, focused signal**. This is the "good prediction" pattern.
- **Diffuse warm coloring throughout** → **the model is averaging over many weak features**. Confidence is usually borderline.
- This map is your **"explainable AI"** answer for presentations and reports — it shows the classifier's reasoning is not a black box.

7. Closest Known Sequences (BLAST-Lite)

What it is

The top-K (default 5) most similar training sequences to your query, ranked by cosine similarity in the TF-IDF feature space.

What it tells you

Each card row contains:

Field	Meaning
Disease badge (color-coded)	Class label of the matching training sequence
Rank (#1, #2, ...)	Position in the similarity list
Similarity %	Cosine similarity (TF-IDF), 0–100%
Sequence preview	First 80 characters of the matched sequence
id	Source NCBI accession or local ID

This is a **classifier-independent sanity check**: if the model says Cancer and the 5 closest known sequences are all Cancer, that's strong corroboration. If the model says Cancer but the closest neighbours are Normal, the prediction is suspect.

How it works under the hood

`src/ml/similarity.py`:

1. **Index build** (one-time, on first server start): TF-IDF-vectorize the entire training corpus, L2-normalize, save as a sparse matrix at `models/similarity_index.npz`.
2. **Query**: vectorize the input sequence with the same TF-IDF transformer, L2-normalize.
3. **Search**: matrix multiply (since vectors are L2-normalized, dot product = cosine similarity).
4. **Rank**: argsort descending, return top-K.

This is "BLAST-lite" because it uses k-mer feature similarity instead of dynamic-programming alignment — much faster, and good enough for routing decisions.

How to use it

1. After a prediction, glance at the disease colors of the top-5 hits.

2. **All same color as your verdict** → strong corroboration ✓
3. **Mixed colors** → look at the highest-similarity hit; if its disease differs from your verdict, dig deeper
4. **Very low similarities (< 20%)** across all 5 hits → your sequence is genuinely novel; trust OOD's HIGH risk warning

💡 Practical tips

- Similarity values **above 80%** mean your sequence is essentially a member of the training set (e.g., a near-duplicate). Predictions are very reliable.
- **30–80%** is the typical "novel but related" zone — predictions are usually correct.
- **Below 30%** indicates a sequence quite different from training data — pair this with OOD's verdict.
- The **preview text** lets you visually compare; if the first 80 chars look totally different, you can't really trust the match even at 30%.
- Click-through to a NCBI page is **not built in** but you can copy the ID and search NCBI manually.

8. 🧬 Open Reading Frames (ORFs)

🎯 What it is

A table of ORFs (start codon → stop codon) found in your sequence on both strands and all three reading frames.

💡 What it tells you

Each row shows one ORF:

Column	Meaning
+ frame 0/1/2 or - frame 0/1/2	Strand (forward + or reverse –) and reading frame
start–end	0-indexed nucleotide positions (end exclusive)
N aa	Predicted protein length in amino acids

Long ORFs (≥ 30 amino acids by default) are listed; shorter ones are suppressed. They are sorted by **protein length descending**, so the most "credible" ORF is at the top.

⚙️ How it works under the hood

`src/core/sequence.py → find_orfs()`:

1. Take the forward strand and its **reverse complement**.
2. For each strand, scan three reading frames (offset 0, 1, 2).
3. Look for **ATG** (start codon).
4. From each ATG, walk 3 bases at a time until a stop codon (**TAA**, **TAG**, **TGA**).
5. If the resulting protein length is $\geq$ **min_protein_length** (default 30 aa), record it as an ORF.

📖 How to use it

1. Look at the longest ORF — that's the most likely real protein-coding stretch.

2. Note its strand: **+** means coded as written, **-** means the gene runs the other direction.
3. Cross-reference with the saliency map: if the saliency hotspots overlap a long ORF, the model's prediction has biological grounding.

💡 Practical tips

- Short sequences (< 200 bp) may have no ORFs ≥ 30 aa — that's normal. The card will display "No ORFs of significant length found."
- **Three random frames per strand × two strands = up to 6 reading frames.** A real gene typically has its longest ORF in just one of them.
- ORFs are not a guarantee of a real gene — they are *candidate* coding regions. Confirmation needs ribosome profiling or experimental data.
- The table **caps display at the top 8 ORFs** in the UI; the API returns up to 10.

9. 📖 API Docs (Top-Right Header Link)

🎯 What it is

A small "API Docs ↗" link in the top-right header. Clicking it opens **Swagger UI** at </docs> in a new tab.

💬 What it tells you

Swagger renders the FastAPI auto-generated documentation:

- Every endpoint, grouped by tag ([meta](#), [sequence](#), [prediction](#), [analysis](#), [chat](#))
- Required and optional parameters
- Pydantic schemas (request and response shapes)
- A **"Try it out"** button on every endpoint that lets you send live requests from the browser

⚙️ How it works under the hood

FastAPI built-in: when you define a route with type-annotated Pydantic models, it auto-generates an **OpenAPI 3.1** specification. Swagger UI is mounted at </docs> and ReDoc at </redoc>.

📖 How to use it

1. Click **API Docs** ↗ in the header (or visit <http://127.0.0.1:8000/docs> directly).
2. Expand any endpoint — say **POST** </predict>.
3. Click **Try it out**.
4. Edit the JSON body in the "Request body" textarea.
5. Click **Execute**.
6. Read the response (status code + body) printed below.

💡 Practical tips

- This is the **fastest way to test** any endpoint without writing curl or Python.
- The **Schemas section at the bottom** documents every Pydantic model — useful when integrating GenomelQ from another app.
- If you change a route in [src/api/app.py](#) and hot-reload is on, the docs update automatically.

- A separate, prettier docs view is available at [/redoc](#).

The Full Analyze Flow (End-to-End)

```

User pastes sequence
↓
[Live Stats]  computeLiveStats() → Length / GC% / Valid (in JS, no server call)
↓
User clicks Analyze Sequence
↓
[POST /predict]  with options + selected backbone
↓
Backend: src/api/service.py → predict_payload()
    |
    ├── validate_sequence() → cleaned DNA + composition
    ├── predict_sequence() → prediction (TFIDF / DNABERT-2 / Ensemble)
    ├── saliency() → per-base scores (if include_explain)
    ├── detect() → OOD risk (if include_ood)
    ├── find_similar() → top-K hits (if include_similar)
    └── find_orfs() → ORF list
↓
JSON payload returned to UI
↓
Alpine state.analyze.result is set
↓
Vue-style reactivity re-renders all 8 result cards
    |
    ├── Verdict + gauge (template)
    ├── Class probabilities (template)
    ├── renderCompositionChart() → Chart.js doughnut
    ├── renderCircularPlot() → Plotly polar
    ├── renderSaliency() → HTML span string
    ├── Similar / ORF tables (template)
    └── (PDF button visible)
  
```

Total time: **~50 ms** for TFIDF, **~1 sec** for DNABERT-2 on GPU.

Cheat Sheet — When to Trust the Verdict

Confidence	OOD Risk	Closest Hits	Verdict trust
> 70%	LOW	All match verdict	 Trust it
> 70%	MEDIUM	Mixed	 Verify
< 50%	HIGH	Random	 Don't trust — sequence likely OOD

Confidence	OOD Risk	Closest Hits	Verdict trust
> 80%	LOW	All match	High confidence — corroborated
> 90%	LOW	All match + saliency hot zone over ORF	Excellent prediction

 Related Documents

- [README.md](#) — high-level project overview
- [DOCS.md](#) — comprehensive technical documentation
- [HOW_TO_RUN_IN_VSCODE.md](#) — VS Code setup and debugging
- [TESTING_GUIDE.md](#) — full test plan for every feature
- [figures/SCREENSHOT_GUIDE.md](#) — how to take screenshots for the README

Last updated as part of the GenomeIQ Step 7 documentation pass.